Lab 1: Delegates and Behavior Injection – Math Operations with Delegates

🧠 Goal

Learn how to define and use delegates in C# to pass behavior as parameters. You'll implement a basic calculator that can perform different operations using delegates.

🛠️ Step-by-Step Instructions

✅ Step 1: Create the Project

1. Open Visual Studio or your preferred C# development environment.
2. Create a new Console App project named `DelegateMathLab`.

✅ Step 2: Define the Delegate

In `Program.cs` (or a new file), define a delegate at the namespace level:

```
delegate double MathOperation(double a, double b);
```

This delegate can reference any method that matches the signature.

✅ Step 3: Implement Operation Functions

Define a few math operations that match the delegate signature:

```
static double Add(double x, double y) => x + y;
static double Subtract(double x, double y) => x - y;
static double Multiply(double x, double y) => x * y;
static double Divide(double x, double y) => y != 0 ? x / y : double.NaN;
```

✅ Step 4: Create the Calculator Class

Create a class that uses the delegate to perform operations:

```
public class Calculator
{
    public double Compute(double a, double b, MathOperation op)
    {
        return op(a, b);
    }
}
```

✅ Step 5: Test with Method Groups

In `Main()`, test your calculator with method references:

```
var calc = new Calculator();

Console.WriteLine($"Add: {calc.Compute(4, 2, Add)}");
Console.WriteLine($"Subtract: {calc.Compute(4, 2, Subtract)}");
Console.WriteLine($"Multiply: {calc.Compute(4, 2, Multiply)}");
Console.WriteLine($"Divide: {calc.Compute(4, 2, Divide)}");
```

✅ Step 6: Test with Lambda Expressions

Now test using lambda expressions directly:

```
Console.WriteLine($"Power: {calc.Compute(2, 3, (x, y) => Math.Pow(x, y))}");
Console.WriteLine($"Modulus: {calc.Compute(10, 3, (x, y) => x % y)}");
```

🌟 Stretch Goal: Add Logging with Another Delegate

Step 7: Define a Logging Delegate

```
delegate void Logger(string message);
```

Step 8: Modify Calculator to Support Logging

```
public class Calculator
{
    public double Compute(double a, double b, MathOperation op, Logger log = null)
    {
        double result = op(a, b);
        log?.Invoke($"Computed {a} and {b}: Result = {result}");
        return result;
    }
}
```

Step 9: Test Logging

```
Logger consoleLogger = msg => Console.WriteLine($"[LOG] {msg}");

var result = calc.Compute(5, 3, Add, consoleLogger);
```

✅ What You Learned

- How to define and use delegates
- How to inject behavior via method groups and lambdas
- How to pass multiple behaviors (operation + logging)

🧪 Ready for More?

Continue to Lab 2 to explore how to use `Func<T>` and lambda expressions with LINQ-style operations.